Introduction to Databases, 2e-B.Sc.
1ST semester 2016–2017, University of Antwerp
Prof. Dr. Toon Calders and M.Sc.Len Feremans

Universiteit Antwerpen

Exam 30 January 2017

- The final exam contains 7 problems and is scheduled for 4 hours.
- Please verify if your question booklet consists of 6 pages (plus 6 pages with supplementary lecture slides) that are printed legibly, else contact your tutor immediately.
- The exam questions address both the theory and exercise parts of the lecture. For questions marked with an asterisk (*), *supplementary lecture* slides are provided at the end of this booklet.
- No electronic devices (such as notebooks, smartphones or smartwatches) are allowed.
- Usage of course material is not allowed.
- Answers *without sufficient details* are *void* (e.g.: you can't just answer "yes" or "no").
- Only the answer booklet will be collected, please do not use the question booklet for answering the questions.
- Take pride in your work: answer clearly and hand in a readable exam copy!

Problem 1: Entity Relational Model and Relational Design (5 points)

Carefully read the following requirement analysis.

A supermarket chain consists of different supermarkets, each one of them identified internally by a 6-character code. For each of these supermarkets, the address consisting of city, postal code, street name, and street number is stored. Whenever a client makes a purchase in one of the supermarkets, a transaction with a unique transaction number has to be recorded in the database. For each of these transactions we store the supermarket where the purchase was made, date and time of the transaction, the payment method (cash, credit card, or debit card), and the list of products purchased. Additionally for each product in a transaction, quantity, unit price, and discount (for instance when a coupon is used) are stored. Next to a product code, each product has a name and a brand. For some transactions we know the client who made the purchase because a loyalty card was used; the other transactions are anonymous. For the transactions in which the loyalty card was used, the customer that made the purchase has to be stored in the database as well. Customers with a loyalty card have a client identifier, a name, an address, and one or more contacts that need to be represented in the database. A contact can be a telephone number or an email address and they are numbered according to the preferential contact method of the user.

An example of a transaction:

A customer makes the following purchase on 26/01/2016 at 11:43 in supermarket Gran Bazar Antwerp (code GBANT):

Product ID	**Product name**	**QTY**	**Unit price**	**Discount**	
P0123	Chips bag 600g 3	1,20	1,20	*1,2*	*(2+1 free)*
P0567	Bread, white	1	2,20	0,00	
P0345	Cereals 1kg	2	3,99	0,30	*(voucher)*
P0678	Milk 1l, 6pack	1	4,33	0,00	

As the customer used his loyalty card, we can connect the transaction to his profile:
Joske Vermeulen with customer identifier 00012345
Trammezandlei 122 in 2900 Schoten
Contact information: 1. Joske.Vermeulen@merksem.be 2. Gaston@antwerpen.be

1. Create an Entity-Relationship diagram that models as faithfully as possible the above description. Clearly indicate entities, relationships, primary keys, cardinalities and participation constraints.
2. (*) Create the necessary create table statements to implement this ER-diagram as a relational database. Define all necessary schema-level constraints (primary keys, foreign keys, domain constraints. Active database instructions such as “On delete/updater cascade/…” are not required).
3. Write a check constraint to assure that discounts are always positive and never exceed the total price for the product. That is: discount should always be between 0 and unit price times quantity.

Problem 2: Triggers and SQL (3 points)

Consider the following (simplified) relational scheme:

User(account, uname)
Group(GID, gname)
Member(account, GID)
Message(MID, account, content)
Likes(account, MID)
Friend(account1,account2)

1. (*) Write a trigger that assures that the Friend relation is symmetric; that is, whenever a tuple (a,b) is inserted in Friend, the tuple (b,a) needs to be inserted in Friend as well.
2. Write the following queries in SQL:
    a. Give all users who *only* like their own messages.
    b. List all messages posted by a user of group "UA professors" that are liked by more than 5 users of the group "UA Students"
    c. List the group with the most messages.t
    d. List the user(s) who have friends in at least 5 different groups.

Problem 3: Relational Algebra (3 points)

For the same database as in problem 2, give relational algebra expressions for the following queries:

a. Give *all* messages that are liked by *all* users of the group "UGent".
b. List all users that are member of both groups "UA" and "UGent", but *not* of "ULB".
c. Give *all* pairs of different groups that have members in common.
d. Give persons that *only* like messages posted by one of their friends.
e. Give *all* messages posted by someone not in the group "UA" that have the same content as a message posted by someone in the group "UA".

Problem 4: Functional dependencies and Normal Forms (3 points)

Consider the following functional dependencies over a relation R(A,B,C,D,E):

{ A→BC, AB→DE, CD→A, D→E }

1. List all candidate keys of R.
2. Which of the following decompositions are lossless?
   a. R_1(ABC) and R_2(CDE)
   b. R_1(BCD) and R_2(ACDE)
3. Which of the following decompositions are dependency-preserving?
   a. R_1(ADE), R_2(ABD), and R_3(ACD)
   b. R_1(ABC) and R_2(ADE)
4. Give a lossless, dependency-preserving decomposition of R into 3NF.
5. Give a lossless decomposition of R into BCNF.

Problem 5: Multivalued dependencies (1 point)

Is the following statement true or false? Proof or give a counter example.

In every relation in which the following functional and MVD dependencies hold:

{ ABC→→F, F→D, FD→E },

also the dependency ABC→→DE *holds.*

Problem 6: XML and XPath (2 points)

Let the following XML document be given:

```
<?xml version="1.0"?>
<catalog>
   <book id="bk101">
      <author>Gambardella, Matthew</author>
      <title>XML Developer's Guide</title>
      <genre>Computer</genre>
      <price>44.95</price>
      <publish_date>2000-10-01</publish_date>
      <description>An in-depth look at creating applications
      with XML.</description>
   </book>
   <book id="bk102">
      <author>Ralls, Kim</author>
      <title>Midnight Rain</title>
      <genre>Fantasy</genre>
      <price>5.95</price>
      <publish_date>2000-12-16</publish_date>
      <description>A former architect battles corporate zombies,
      an evil sorceress, and her own childhood to become queen
      of the world.</description>
   </book>
   <book id="bk103">
      <author>Corets, Eva</author>
      <title>Maeve Ascendant</title>
      <genre>Fantasy</genre>
      <price>5.95</price>
      <publish_date>2000-11-17</publish_date>
      <description>After the collapse of a nanotechnology
      society in England, the young survivors lay the
      foundation for a new society.</description>
   </book>
…
```

Give XPath expressions for the following queries:

1. Give all book elements of genre "`Fantasy`" written by author "`Ralls, Kim`".
2. Return all books that do not have an author.
3. Return all books that have multiple genre elements.
4. Find the cheapest book by "`Corets, Eva`"

Problem 7: Datalog (3 points)

When combining negation with recursion in datalog or recursive SQL it is important that the query/datalog program to be evaluated is stratified.

a. Explain what is a stratified datalog program and how the dependency graph can help us to decide if a program is stratified.
b. Is the following program stratified? What does the intensional relation LowRisk express?
Parent(x,y) means that x is the parent of y.
Male(x), Female(x) means that x is a male, respectively female person.
ColorBlind(x) means that x suffers from color blindness.

```
Father(x,y) :- Parent(x,y), Male(x).
MaleAncestor(x,y) :- Father(x,y).
MaleAncestor(x,z) :- MaleAncestor(x,y), Parent(y,z).
LowRisk(x) :- Female(x).
ColorBlindMaleAncestor(y) :- MaleAncestor(x,y), ColorBlind(x).
LowRisk(x) :- Male(x), not(ColorBlindMaleAncestor(x)).
```

c. Show the different execution steps of this program on the following extensional relations:

Parent	
Jan	Piet
Piet	Rita
Rita	Geert
Rita	Jef
Jan	Klaas
Klaas	An
Klaas	Suzan

Male
Jan
Piet
Geert
Jef
Klaas

Female
Rita
An
Suzan

ColorBlind
Piet

Supplementary material:

Defining Foreign Keys

```
CREATE TABLE Orders
(
        O_Id int NOT NULL,
        OrderNo int NOT NULL,
        P_Id int,
        PRIMARY KEY (O_Id),
        FOREIGN KEY (P_Id) REFERENCES Persons(P_Id)
)
```

Does not have to be the same attribute name. But number and types need to match

Could also be multiple attributes

Enforces that every Orders.P_Id occurs in the Persons table in the column P_Id.

Universiteit Antwerpen

Value-based CHECK

CHECK constraints allow us to express additional value restrictions for attributes (not only within keys)

```
CREATE TABLE MovieStar (
        name CHAR(30) PRIMARY KEY,
        address VARCHAR(255),
        gender CHAR(1) CHECK (gender IN ('M','F')),
        birthdate DATE );

CREATE TABLE Studio (
        name CHAR(30) PRIMARY KEY,
        address VARCHAR(255),
        presCertN INT CHECK (presCertN >= 1000000) REFERENCES MovieExec(certN) );
```

Universiteit Antwerpen

Triggers in SQL

General syntax:

```
CREATE TRIGGER <trigger_name>
 {BEFORE|AFTER}  {{INSERT|DELETE|UPDATE} [OF <row_list>]}+  ON <table_name>
 [REFERENCING  [NEW ROW|TABLE AS <new_name>] [OLD ROW|TABLE AS <old_name>]]
 [FOR EACH ROW|STATEMENT [WHEN (<trigger_condition>)]]
BEGIN
 <trigger_body>
END;
```

Universiteit Antwerpen

* SQL'99 allows both ROW and TABLE/STATEMENT level triggers. Oracle allows only ROW level, hence the keyword ROW is omitted.

Examples

- Likes(Drinker, Beer)
- Serves(Pub, Beer)
- Visits(Drinker, Pub)

Give all drinkers who do not like "Guiness"

$(\pi_{Drinker}\ Likes \cup \pi_{Drinker}\ Visits) - \pi_{Drinker}\ \sigma_{Beer = "Guiness"}\ Likes$

All drinkers

Drinkers who like Guiness

Universiteit Antwerpen

Examples

- Likes(Drinker, Beer)
- Serves(Pub, Beer)
- Visits(Drinker, Pub)

Give all drinkers who visit a bar that serves "Guiness"

$$\pi_{Drinker}\ \sigma_{GPub=Pub}\ (Visits \times (\rho_{GPub/Pub}\ \sigma_{Beer="Guiness"}\ Serves))$$

Give all drinkers who visit a bar that serves a beer they like

$$\pi_{Drinker}\ \sigma_{VP=SP\ \wedge\ SB=Beer\ \wedge\ Drinker=VD}\ (Likes \times \rho_{SP/Pub,\ SB/Beer}\ Serves \times \rho_{VD/Drinker,\ VP/Pub}\ Visits)$$

=

$$\pi_{Drinker}\ \sigma_{SB=Beer\ \wedge\ Drinker=VD}\ [Likes \times \sigma_{VP=SP}\ (\rho_{SP/Pub,\ SB/Beer}\ Serves \times \rho_{VD/Drinker,\ VP/Pub}\ Visits)]$$

Universiteit Antwerpen

Keys & Functional Dependencies

A set of one or more attributes $\{A_1, A_2, ..., A_n\}$ is called a **key** for a relation *R* if:

(1) $\{A_1, A_2, ..., A_n\}$ functionally determines *all attributes* of *R*.

(2) *No proper subset* of $\{A_1, A_2, ..., A_n\}$ functionally determines all attributes of *R*.

→ In other words, a key must be a *minimal set of attributes* for which property (1) holds.

A set of attributes $\{A_1, A_2, ..., A_n\}$ that contains a key of relation *R* is called a **superkey** of *R*.

→ Every key is also a superkey.

Universiteit Antwerpen

A Complete Set of Inference Rules

Armstrong's Axioms:

The following three axioms allow for deriving any FD that follow from a given set of FD's.

1. Reflexivity:
   If $\{B_1, B_2, ..., B_m\} \subseteq \{A_1, A_2, ..., A_n\}$,
   then $A_1, A_2, ..., A_n \rightarrow B_1, B_2, ..., B_m$. ($\rightarrow$ *"trivial FDs"*)
2. Augmentation:
   If $A_1, A_2, ..., A_n \rightarrow B_1, B_2, ..., B_m$,
   then $A_1, A_2, ..., A_n, C_1, C_2, ..., C_k \rightarrow B_1, B_2, ..., B_m, C_1, C_2, ..., C_k$,
   for any set of attributes $C_1, C_2, ..., C_k$. ($\rightarrow$ *"non-trivial FDs"*)
3. Transitivity:
   If $A_1, A_2, ..., A_n \rightarrow B_1, B_2, ..., B_m$ and $B_1, B_2, ..., B_m \rightarrow C_1, C_2, ..., C_k$,
   then $A_1, A_2, ..., A_n \rightarrow C_1, C_2, ..., C_k$. ($\rightarrow$ *"closure algorithm"*)

Universiteit Antwerpen

Normal Forms of Relations

In the following, let *X* be a *set of attributes* and *Y* be a *single attribute* of a relation *R*, and let *F* be a given set of FD's on *R*.

R is in **Third Normal Form** (3NF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R or Y is an attribute of a key of R.

R is in **Boyce-Codd Normal Form** (BCNF), if for every non-trivial FD $X \rightarrow Y$ in F it holds that X is a superkey of R.

Universiteit Antwerpen

Algorithm for Computing a Dependency-Preserving Decomposition in 3NF

Given a relation *R* with attributes *A*, a set of keys *K*,
and a *minimal basis* of FD's *F*.

Decompose R into:

- $R_1 = \pi_{K'}(R)$ where K' is a key in K which becomes key of R_1;
  (if there are multiple keys in R, choose one)
- for each $X \rightarrow Y$ in F,
  the projection $R_{i+1} = \pi_{X \cup Y}(R)$ forms a new relation
  in the decomposition with key X.

Universiteit Antwerpen

Algorithm for Computing a Decomposition in BCNF (not dependency-preserving!)

Given a relation *R* with attributes *A*, a set of keys *K*,
and a set of FD's *F*.

If there is a non-trivial FD $X \rightarrow Y$, where X is not a superkey of R,
then decompose R into:

- $S = \pi_{X^+(F)}(R)$ with the new key X;
- $T = \pi_{X \cup (A - X^+(F))}(R)$ by taking over all remaining keys from R.

Then recursively decompose S and T into BCNF using the
remaining (non-violated) FD's in F.

Universiteit Antwerpen

Multivalued Dependency

A **multivalued dependency** (MVD), denoted as

$A_1,A_2,...,A_n \rightarrow\rightarrow B_1,B_2,...,B_m$

holds for a relation *R* if, for each pair of tuples *t*, *u* of *R*, which agree on all the A's, there is a tuple *v* in *R* that also agrees

- with both *t* and *u* on the A's;
- with *t* on the B's;
- with *u* on *all other attributes* of *R* not among the A's and B's.

Example:

	starName	→→	city,street		title,year
	A's		B's		others
t	a_1		b_1		c_1
	Ford		Malibu, Maple Str.		Star Wars, 1977
v	a_1		b_1		c_2
	Ford		Malibu, Maple Str.		Star Wars, 1980
u	a_1		b_2		c_2
	Ford		Hollywood, Mulh. Dr.		Star Wars, 1980

Univ
Antwerpen